PLATFORMCON SÃO PAULO · 07/11/2026 · WORKSHOP · 60 MIN

Quando o usuário da plataforma é um agente

Do trace de uso à verificação no kernel, com MCP, CI e eBPF

Fabio Wakim Trentini

github.com/trentas

Kit do workshop

github.com/trentas/platformcon-workshop-kit

ABERTURA

Antes de começar

- ✓ Kit clonado e preflight rodado sem erro (make preflight).
- ✓ Um agente de IA compatível com MCP, aberto e autenticado.
- ✓ Dois terminais abertos na pasta do kit, na sua máquina Linux ou VM.
- Wi-Fi: informado na sala
- </> Kit do workshop: `github.com/trentas/platformcon-workshop-kit`

Sem ambiente pronto?

Faça em dupla com quem está ao lado.

O kit traz capturas prontas para o bloco 3.

ABERTURA

O que vamos fazer aqui

Desde julho, um loop que lê as sessões dos agentes de IA que usam um produto em produção abriu metade das issues desse produto. Hoje vamos montar uma versão pequena desse ciclo, com os seus agentes e as suas máquinas.

Antes, alguns minutos sobre como o ciclo funciona no caso de origem, o que ele já encontrou e onde ele erra.

O CASO DE ORIGEM

De onde vêm os exemplos

Os exemplos vêm do WhyStack, uma plataforma de diagnóstico de infraestrutura que desenvolvo desde junho de 2026. Ele aparece nesta abertura como caso de estudo; o material do workshop é aberto e não depende dele.

Medida	Valor
Commits com coautoria de um agente de código	921 de 940 (98%)
Issues abertas pelo loop de reflexão	448 de 877 (51%)
Issues do loop fechadas como concluídas	322 de 418 fechadas
Tempo mediano até concluir uma issue do loop	2,6 dias

Contagem no git e no GitHub, de 01/06 a 15/10/2026.

O CASO DE ORIGEM

Dois ciclos sobre os mesmos traces

Sessão de um agente

Um agente de IA investiga um problema em produção. O trace guarda a forma das chamadas.

Juiz com rubrica

Um LLM com prompt fixo classifica o que a sessão mostra.

Auto-learn: fatos sobre o ambiente

Qual repositório é de qual serviço, quem chama quem, como um serviço é chamado em cada fonte. Vira dado do cliente e volta na próxima sessão, sem código novo.

Auto-improve: defeitos do produto

Uma tool que levou o agente a uma conclusão errada, ou que faltou. Vira issue com a label reflection, para uma pessoa fazer a triagem.

O que o produto aprende

Fato	De onde vem	Como volta
Serviço → repositório	Config do Sonar, commit de deploy, confirmação de uma pessoa	A tool resolve o repositório sem nova busca
Quem chama quem	Spans, DNS, catálogo do Datadog	Grafo de dependências e resumo da sessão
Particularidades do ambiente	O juiz, a partir de sessões anteriores	Nota no resumo do início de cada sessão

Grau da prova

Declarado, derivado ou inferido. Palpite não rebaixa prova; o que uma pessoa confirmou fica.

Veto de uma pessoa

O time confirma ou veta cada fato na tela de insights. O veto vale para as escritas seguintes.

Entre clientes

Só com opt-in do cliente, anonimizado. O aprendizado de cada cliente não depende disso.

O CASO DE ORIGEM · AUTO-IMPROVE

Do trace à issue, em seis etapas

01 **Captura**
Cada sessão MCP vira um trace, já sem dado pessoal: a redação acontece antes de o dado sair do ambiente do cliente.

02 **Amostragem**
Sessão com erro, escrita, 8 ou mais chamadas ou forma de desvio é sempre avaliada. O resto, por amostra.

03 **Juiz**
Um LLM com rubrica versionada recebe o trace marcado como dado não confiável e classifica cada achado.

04 **Roteamento**
Fato do ambiente vira dado aprendido. Defeito do produto vira issue, com o observado separado da hipótese.

05 **Deduplicação e veto**
Achados parecidos se juntam numa issue só. O que o cliente vetou é filtrado antes.

06 **Gate de avaliação**
Mudança na rubrica ou no modelo passa por um replay sobre traces sintéticos antes de valer.

Uma pessoa faz a triagem de toda issue e confere a causa no código antes de implementar.

O que o loop encontrou, por tipo

Sinal	O que significa	Issues
Sinal falso	Uma tool levou o agente a uma conclusão errada	150
Encadeamento manual	O agente combinou várias tools para uma pergunta só	96
Falta no connector	Faltou um dado que a integração deveria expor	54
Fato de catálogo	Fato sobre o ambiente, registrado também como issue	53
Bloqueio	O agente pediu ao usuário o que a plataforma deveria dar	42
Desvio	O agente contornou uma tool que faltava	29
Pista falsa	Uma pista custou tempo e não se confirmou	14

A primeira rodada, em 29/07, abriu 145 issues de uma vez. O tipo mais comum é o tema do bloco 1: uma resposta que leva o agente a concluir errado.

Contagem pelo tipo no título das issues, 15/10/2026

O CASO DE ORIGEM · AUTO-IMPROVE

Três achados e o que mudou

Seis issues, um defeito

Seis tools responderam “nada encontrado” com a fonte fora do ar, e o juiz abriu seis issues de sinal falso.

A regra de declarar a fonte não lida já existia, mas estava aplicada em 3 de 190 tools. Passou a valer num ponto único, para todas.

“Saudável” com milhares de erros

Uma investigação respondeu “saudável, 0% de erro” numa janela com dezenas de milhares de respostas 4xx. O agente precisou de cerca de 20 consultas manuais.

A tool passou a ler a série por intervalo, com a taxa de erro dos logs ao lado da do APM.

Fechada sem código

Depois de cerca de 20 consultas sobre filas, o juiz pediu uma tool nova.

A tool já existia e tinha sido recusada porque a conta não tinha Agent conectado. O ajuste era de configuração, e a issue foi fechada como não planejada.

O CASO DE ORIGEM · AUTO-IMPROVE

Onde o loop erra

A causa apontada erra

Numa revisão de 5 achados corrigidos, o sintoma estava certo nos 5 e a causa, errada em 4. Por isso a issue separa o observado da hipótese.

O próprio loop pode estar cego

Um erro chegou ao juiz cortado em 119 caracteres. Ele culpou o connector; quem cortava era o resumidor do trace.

Agente competente esconde o problema

Quando o agente contorna uma tool ruim, a sessão termina bem. O sinal está na forma da sessão, não no resultado.

Nem toda issue vira código

Das 93 fechadas como não planejadas, cerca de 58 eram duplicatas. As outras foram recusas: a tool já existia, ou a causa não se confirmou.

Por isso o loop abre issue e nunca PR: quem decide o que vira código é uma pessoa.

HOJE

O que vamos reproduzir hoje

BLOCO 1

Agente em produção

- consulta a plataforma via MCP
- o trace guarda a forma da chamada
- um avaliador lê os traces

Humano: triagem

BLOCO 2

Agente de código

- a issue vira tarefa
- o agente escreve a correção
- o CI barra erros conhecidos

Humano: merge

BLOCO 3

Verificação do deploy

- ptop coleta com eBPF
- diff com a versão anterior
- v1 × v1 estima o ruído

Humano: release

Depois do deploy, os traces de produção da nova versão entram no ciclo seguinte.

HOJE

Os 60 minutos



Minutos	Bloco	Formato
0–10	Abertura: o caso de origem e o que vamos fazer	Instrutor
10–27	Bloco 1: agentes como usuários de um servidor MCP	Hands-on
27–32	Bloco 2: verificações de CI para código escrito por agentes	Demonstração
32–55	Bloco 3: comparação de versões com eBPF (ptop)	Hands-on
55–60	Tema comum, como aplicar e material	Instrutor

BLOCO 1 · HANDS-ON · 17 MIN

Agentes como usuários de um servidor MCP

Todos conectam o próprio agente ao mesmo servidor de exemplo. O servidor registra cada chamada de tool, e é a partir desses registros que vamos encontrar um problema na plataforma.

Comandos completos: README do kit, seção Bloco 1

BLOCO 1 · PASSO 1

Conectar o agente ao servidor

Endereço do servidor

`https://lesser-goldsmith-numerous.ngrok-free.dev/mcp`

Claude Code, no terminal

```
claude mcp add --transport http
plataforma <endereço>
```

Cursor: ~/.cursor/mcp.json

```
{"mcpServers": {"plataforma": {
  "url": "<endereço>"}}}
```

VS Code: .vscode/mcp.json

```
{"servers": {"plataforma": {
  "type": "http",
  "url": "<endereço>"}}}
```

Abra o agente numa pasta vazia, fora do kit, e reinicie-o depois de configurar.

Para conferir: o agente deve listar as 3 tools do servidor “plataforma”.

BLOCO 1 · PASSO 2

A mesma pergunta para todos

> O serviço pagamentos teve erros na última hora? Use o servidor plataforma.

01 Abra uma sessão nova do agente.

02 Faça só esta pergunta, sem complementar.

03 Anote a resposta, sem corrigir nem insistir.

A pergunta também está no README do kit, para copiar

O que o servidor registra

```
{"sessao": "s-7f3a", "seq": 1,
 "tool": "buscar_erros",
 "args": {"servico": "pagamentos",
          "janela_min": 60},
 "resultado": {"status": "ok", "itens": 0},
 "duracao_ms": 41}
```

Registra

Nome da tool, argumentos estruturados, status, quantidade de itens, ordem e duração das chamadas.

Não registra

O prompt, a resposta do agente e o conteúdo de argumentos de texto livre, que viram tamanho e hash.

O servidor nunca vê a resposta do agente. Ele vê a forma da sessão: quais tools, em que ordem e com que resultado.

BLOCO 1 · PASSO 3

As sessões da sala, agrupadas pela forma

Sequência de tools	Leitura
buscar_erros → fim	Parou no resultado vazio
buscar_erros → listar_servicos → fim	Conferiu o nome do serviço e parou
buscar_erros → listar_deploys → …	Procurou outra explicação

Levante a mão quem recebeu a resposta “não houve erros”. Compare com a primeira linha do painel.

BLOCO 1 · PASSO 4

O avaliador classifica cada sessão

Categoria	Pergunta que o avaliador responde	Exemplo na sala
Sinal falso	Algum resultado levou o agente a uma conclusão que os dados não sustentam?	Lista vazia lida como “sem erros”
Pista falsa	O agente seguiu uma explicação que não se confirmou?	Deploy recente tratado como causa
Encadeamento manual	O agente precisou combinar várias tools para uma pergunta só?	listar_servicos, buscar_erros e listar_deploys
Bloqueio	O agente pediu ao usuário algo que a plataforma deveria dar?	Pediu o nome exato do serviço

Rubrica e prompt do avaliador: avaliador/rubrica.md, no kit

BLOCO 1 · PASSO 4

O rascunho de issue

buscar_erros devolve lista vazia quando a fonte de logs falha

Observado (verificável nos traces)

- n de m sessões chamaram buscar_erros e receberam 0 itens com status ok
- k dessas sessões terminaram sem outra chamada
- a telemetria da plataforma mostra os logs fora do ar durante as sessões

Hipótese (a confirmar no código)

- o vazio vem da falha da fonte, não da ausência de erros

O observado vem com número e pode ser conferido no trace.

A causa vem marcada como hipótese, pelo motivo visto na abertura: no caso de origem, ela errou em 4 de 5 achados revisados.

Quem abre a issue é uma pessoa, depois da triagem.

BLOCO 1 · PASSO 5

O servidor passa a declarar o que não leu

v1

```
buscar_erros(servico="pagamentos")
→ {"erros": []}
```

v2

```
buscar_erros(servico="pagamentos")
→ {"erros": null,
   "fontes": {"logs": "indisponivel"},
   "aviso": "logs não lidos"}
```

Faça a mesma pergunta numa sessão nova. Não é preciso mudar a configuração: o servidor já está na v2.

O painel mostra as formas das sessões da v1 e da v2 lado a lado.

BLOCO 1 · RESUMO

Três conceitos do bloco 1

Forma, não conteúdo

O trace registra quais tools, em que ordem e com que resultado. Isso basta para achar o problema sem guardar o que o usuário escreveu.

Observado e hipótese

O rascunho separa o que pode ser conferido no trace da causa suposta. A triagem confere a causa antes de alguém implementar.

Fonte declarada

Lista vazia quer dizer “li e não havia nada”. O que não foi lido vem como null, com a fonte declarada.

BLOCO 2 · DEMONSTRAÇÃO · 5 MIN

Verificações de CI para código escrito por agentes

A correção do problema do bloco 1 vai ser escrita por um agente de código. Este bloco mostra o que impede o mesmo erro de voltar na próxima tool que ele escrever.

Demonstração no repositório do kit: pasta guardas/

BLOCO 2

Verificação no código-fonte

servidor/tools/deploys.ts

```
async function listarDeploys() {
  try {
    return await backend.deploys()
  } catch {
    return []
  }
}
```

Saída do CI

```
$ make guardas
guardas/catch-vazio
  FALHA servidor/tools/deploys.ts:5
  catch devolve [] sem declarar
  a fonte que falhou
  use: semFonte("deploys", erro)
```

A mensagem é escrita para quem vai corrigir, que muitas vezes é o próprio agente: diz o que fazer, não só o que falhou.

Demonstração no repositório do kit: pasta guardas/

BLOCO 2

Teste da própria verificação

guardas/catch-vazio.test.ts

```
test("a varredura leu arquivos", () ⇒ {
  const r = varrer("servidor/tools")
  expect(r.arquivos).toBeGreaterThan(0)
  expect(r.violacoes).toEqual([])
})
```

Se a pasta das tools muda de lugar, a varredura não encontra nenhum arquivo e passa sem ter verificado nada.

O teste exige que ela tenha lido pelo menos um arquivo antes de aceitar zero violações.

No caso de origem, a mesma verificação também precisou ignorar comentários: um comentário que citava o padrão proibido disparava a regra.

Demonstração no repositório do kit: pasta guardas/

BLOCO 2

Dois custos de trabalhar com agentes

Problema	Medida no caso de origem	Mudança
Arquivo de instruções do agente (CLAUDE.md)	154 KB, cerca de 40 mil tokens em toda sessão	Regra de tamanho por seção; detalhes em arquivos lidos quando necessário. Hoje: 47 KB.
Changelog editado por PRs paralelas	5 PRs custaram 8 execuções de CI	Um arquivo por PR, montado no changelog na hora do release.

Quando quem programa é um agente, o repositório funciona como a plataforma dele: o arquivo de instruções faz o papel da documentação, e o CI garante as convenções.

BLOCO 3 · HANDS-ON · 23 MIN

Comparação de versões com eBPF (ptop)

Um serviço Go tem duas versões. A v2 abre uma conexão nova a cada request em vez de reaproveitar o pool. Vamos medir a diferença no kernel e compará-la com a variação entre duas execuções da mesma versão.

Comandos completos: README do kit, seção Bloco 3

BLOCO 3 · PASSO 1

O que o ptop vai coletar

O preflight já aplicou as capabilities ao binário do kit. Confira antes de capturar:

```
$ ./bin/ptop --caps
```

Todos os coletores devem aparecer como “will run”.

Saída esperada (trecho)

```
collectors, with the privileges this process holds
  ✓ cpu          will run
  ✓ heap         will run
  ✓ network      will run
  ✓ syscalls     will run
```

```
sudo setcap cap_bpf,cap_perfmon,cap_sys_admin,cap_dac_read_search,cap_sys_ptrace+ep ./bin/ptop
```

O comando que o preflight rodou. cap_sys_admin equivale a quase root: conceda numa VM de laboratório.

Comandos completos: README do kit, seção Bloco 3

BLOCO 3 · PASSO 2

Capturar a v1

Terminal 1

```
# sobe a v1 com carga constante
$ make v1
```

Terminal 2

```
# captura 60 segundos
$ make captura NOME=v1-a
```

O que o make captura executa

```
timeout -s INT 60 ./bin/ptop --pid $(cat run/servico.pid) \
    --serve unix:///tmp/ptop.sock --export
```

O --export grava o mesmo stream em JSONL, em capturas/v1-a.jsonl. O terminal 1 mostra a latência p50 e p99 a cada 10 segundos: anote os valores.

BLOCO 3 · PASSO 3, DURANTE A CAPTURA

Um leitor sem privilégio

Terminal 3

```
$ grep CapEff /proc/self/status
CapEff:  0000000000000000
$ ./bin/assinante unix:///tmp/ptop.sock
{"meta":{"target":{"mode":"TARGET_MODE_PID","pid":4242,"probes":[…]}}}
{"event":{"category":"CATEGORY_SYSCALL",…}}
```

O coletor tem as capabilities. Quem lê o stream não precisa de nenhuma.

tcp:// sem TLS só sobe com --serve-insecure. Na rede, o ptop usa mTLS.

O socket unix só aceita o seu usuário (permissão 0600).

A primeira mensagem diz quais coletores estão ativos.

Comandos completos: README do kit, seção Bloco 3

BLOCO 3 · PASSO 4

Capturar a v2 e repetir a v1

Terminal 1 e terminal 2, em sequência

```
$ make v2                # para a v1 e sobe a v2 com a mesma carga
$ make captura NOME=v2
$ make v1                # volta para a v1
$ make captura NOME=v1-b  # segunda captura da v1
```

A latência

Compare o p50 e o p99 que o terminal 1 mostra na v1 e na v2. A v2 foi feita para mudar pouco a latência.

Por que duas capturas da v1

A diferença entre v1-a e v1-b mostra quanto o resultado varia sem nenhuma mudança de código.

Comandos completos: README do kit, seção Bloco 3

BLOCO 3 · PASSO 5

Comparar as capturas

```
$ make comparar A=v1-a B=v2    # mudança de código
$ make comparar A=v1-a B=v1-b  # ruído
```

Métrica	De onde vem	O que a v2 altera
connect() por request	syscalls, no ptop	Passa de perto de 0 para 1
socket() e close() por request	syscalls, no ptop	Acompanham o connect()
Latência p50 e p99	gerador de carga	Pouco

Uma diferença entre v1 e v2 só conta se for maior que a diferença entre v1-a e v1-b.

Comandos completos: README do kit, seção Bloco 3

BLOCO 3 · PASSO 6

Capabilities incompletas

```
$ sudo setcap cap_bpf,cap_perfmon+ep ./bin/ptop
$ ./bin/ptop --caps
$ make captura NOME=v1-parcial
$ make comparar A=v1-a B=v1-parcial
```

Saída do make comparar (trecho)

```
coletores diferentes entre v1-a e v1-parcial:
  heap     v1-a: active (eBPF)  v1-parcial: failed
  syscalls v1-a: active (eBPF)  v1-parcial: failed
  (…)
números não comparados:
```

Até a v1.4, o README do ptop indicava só cap_bpf e cap_perfmon. Com esse conjunto, parte dos coletores não roda, e o valor deles aparece como zero. Zero é uma medida: diz que o processo não fez aquilo.

Hoje o --caps avisa antes, e a primeira mensagem do stream informa o estado de cada coletor: active, disabled, failed ou unsupported.

Ao terminar: make preflight, para restaurar o conjunto completo de capabilities

BLOCO 3 · RESUMO

Quatro conceitos do bloco 3

O que a latência não mostra

A comparação no kernel mostra o que mudou por request, mesmo quando a latência quase não muda.

Ruído antes de diferença

Uma diferença só conta se for maior que a variação entre duas execuções da mesma versão.

Instrumentação antes de números

Antes de comparar valores, confira se os mesmos coletores estavam ativos nas duas capturas.

Privilégio só no coletor

Quem lê o stream não precisa de capabilities. O acesso ao kernel fica num processo só.

ENCERRAMENTO

O que os três blocos têm em comum

Camada	Problema	O que foi feito
Interface do agente	Com a fonte fora do ar, a tool devolvia lista vazia, e o agente concluía “sem erros”.	O que não foi lido vem como null, com a fonte declarada.
CI	A verificação passava quando não encontrava nenhum arquivo para varrer.	Um teste exige que a varredura tenha lido arquivos.
Kernel	Com capabilities incompletas, os coletores parados apareciam como zero.	O --caps avisa antes, e o stream traz o estado de cada coletor.

Nas três camadas, a correção foi a mesma: dizer o que não foi lido ou executado, em vez de devolver um resultado vazio ou zerado.

ENCERRAMENTO

Para montar um ciclo parecido na sua plataforma

01 Registre a forma das chamadas de tool: nome, argumentos estruturados, quantidade de itens e ordem. Texto livre vira tamanho e hash.

02 Rode um avaliador com rubrica sobre os traces e gere rascunhos que separem o observado da hipótese.

03 Faça a triagem com uma label própria no issue tracker e acompanhe quantos rascunhos são descartados.

04 Compare cada deploy com a versão anterior e com a variação entre duas execuções iguais, conferindo antes a instrumentação.

ENCERRAMENTO

Material

</>

Kit do workshop

Servidor MCP de exemplo, avaliador e rubrica, verificações de CI, serviço v1/v2, script de comparação e as capturas de hoje.

`github.com/trentas/platformcon-workshop-kit`

ptop

Coletor eBPF de código aberto (MIT), com interface de terminal e stream gRPC.

`github.com/trentas/ptop`

20-Factor App

Metodologia aberta que estende o 12-Factor para aplicações com IA.

`github.com/trentas/20-factor`

ENCERRAMENTO

Perguntas

Avaliação do workshop

Fabio Wakim Trentini

github.com/trentas

Kit

github.com/trentas/platformcon-workshop-kit